Agentforce for Setup v2: Core Understanding, From First Principles

Read this before the architecture PDF. The other document tells you *what* the system is. This one teaches you *why every piece exists*, what it holds, what it can break, and how to reason about it, so you can answer a question you've never seen before.

> **Confidential study material.** Internal component names appear here. In the interview, speak at the pattern level.

Evidence tags used for system-specific claims: [C] verified in Core code · **[O]** verified in an off-core service repo · **[S]** stated in Slack, not in code · **[I]** inference · **[N]** searched for, not found. General concepts (what a JWT is, what RAG is) carry no tag: they are standard engineering knowledge.

How to read it: Part A gives the big idea. Part B teaches the building blocks. Part C takes each component apart. Part D replays one request in slow motion. Parts E–G cover design reasoning, failure diagnosis, and self-tests. Part H is a glossary.

Part A. The problem and the big idea

A1. What problem is this solving?

Salesforce *Setup* is where administrators configure an organisation: create users, groups, permission sets, sharing rules, flows, objects, fields. There are thousands of distinct tasks. Today an admin clicks through screens, or reads help articles, and often gets it wrong because the task spans several screens.

The product idea: let the admin say it in plain language ("Create a public group Sales Ops and add Ravi and Meera") and have an agent do it, safely.

"Safely" is the whole difficulty. Setup changes are *privileged writes*: wrongly granting access, deleting a field, or changing a sharing rule can expose data or break the org. So the system is not really "a chatbot that calls an API." It is **a way to let a probabilistic model influence privileged actions without letting it decide them**.

Keep that sentence. Almost every design choice below is a consequence of it.

A2. Why v1 hit a wall

v1 worked like this: a model read some hand-written *instructions* about a *topic* ("user management"), and picked from a *small list of hand-built actions*. The pilot already had **45 bespoke actions** [HLD]. Each one is a piece of Java someone wrote for one task (Freeze User, Create Flow, Create Object).

That scales linearly with engineering effort, and Setup has far more tasks than engineers can hand-build. The design doc calls this the **long tail**: a few popular tasks are easy to hard-code; the thousands of rare ones are not.

There was a second problem: **no planning and no clarification**. v1 picked the likeliest action and lived with being wrong sometimes. It could not say "which user?" before acting.

A3. The idea of v2, in five sentences

1. **Knowledge as data, not code.** Describe *how to do a task* in a Markdown document (an NLD). Adding a capability means adding a document, not writing Java.
2. **Retrieve before acting.** The agent first looks up the relevant documents, then plans.
3. **Plan, then show the plan.** The model produces a step-by-step plan; a human sees and approves it.
4. **Execute with a small set of generic tools.** Instead of 45+ special actions, a few broad actions (create/read/update/delete a record) carry out whatever the plan says.
5. **Hard wall between thinking and doing.** The part that reasons never directly holds the power to change things; changes go through code that checks permissions and asks for confirmation.

If you can say these five sentences from memory, you can reconstruct most of the architecture.

Part B. The building blocks

Each block has four parts: **What it is · Why it exists · How it appears here · What breaks.**

B1. A language model, and why you cannot let it act directly

What: A model that takes text in and predicts text out. Given a request and some context, it writes a plausible response.

Why this matters for actions: "Plausible" is not "correct." A model can invent a user that doesn't exist, mis-spell a field name, skip a step, or follow an instruction hidden in text it was shown. It is also not repeatable: the same input can give different outputs.

Consequence for design: treat the model as an *advisor that proposes*, and put deterministic code between the proposal and the side effect. Deterministic code can check "does this user exist?", "is the caller allowed?", "is the call shaped correctly?", and "did a human approve?". Models cannot be trusted to do those checks on themselves.

Here: the model classifies the request, picks documents, drafts a plan, and phrases replies. The Core code validates and executes. [C/O]

What breaks: if the model's output is executed without a check, one hallucination becomes a production change.

B2. Tools / actions

What: A named, schema-typed function the agent is allowed to call: `createGroup(name, description, …)`. The model outputs "call `createGroup` with these arguments"; *code* actually runs it.

Why: it narrows what the model can do to a menu, and the menu can be validated.

Here: the actions are `CreateGroup`, `ManageUserAssignments`, `QueryDataSource`, and others. Each has an input schema that rejects unknown fields (`unevaluatedProperties: false`) [C]. Each has a permission check. Some are blocked from the public API and are reachable only through the agent path [C].

Bespoke vs generic: a *bespoke* action does one named task. A *generic* action (`ExecuteEntityCreate`) takes "which API family, which entity, which fields" as parameters, so one tool covers many tasks. Bespoke = narrow, testable, bounded. Generic = wide coverage but a bigger capability surface. v2 moves toward generic, so safety shifts from "the tool is narrow" to "the checks around the tool are strong."

What breaks: a generic tool without a strong gate is close to giving the model root on the API.

B3. The agent loop: ReAct vs a graph (bridge to what you already know)

ReAct loop: the model repeatedly does *Reason → Act (call a tool) → Observe the result*, deciding each next step itself, until done. It is flexible but loose: control flow lives inside the model's head, so it is hard to guarantee order, hard to pause, and hard to test.

Agent graph: you *draw* the control flow in advance as **nodes** (steps) and **edges** (allowed transitions) with shared **state**. The model works *inside* a node; the graph decides what comes next. If you know **LangGraph** (you do, from ResolveFlow), this is the same idea: nodes, edges, state, and the ability to end a turn and resume.

Here: the reasoner runs a graph with four nodes: **Knowledge** (retrieve), **Assistant** (build the plan), **Executor** (run it), **Discovery** (find records/metadata). Invariants I-1..I-6 on the graph are checked by a pre-push hook [C].

Why a graph: the design doc says it moved *away from a "simplistic ReAct loop"* to get reliability: the order "retrieve → plan → approve → execute" is enforced by structure, not hoped for.

What breaks: the more structure, the more latency. The design doc admits the graph runtime was **10–20% slower** than the old ReAct planner (Jan 2026; not re-verified).

B4. State and sessions

What: a conversation is a multi-turn thing. Between turns the system must remember: what was said, what plan exists, whether it was approved.

Why it is subtle here: the *approval pause* ends the turn. The system must park the plan, return control to the user, and later resume when the approval arrives as a brand-new message.

Here: session state lives in **Redis** next to the session engine; the graph's state lives in **Redis** next to the reasoner [O]. Whether the approval state has a time-to-live is **[N]**.

What breaks: if state is lost (a restart), a half-finished plan is lost; if state never expires, stale plans can be approved later against a changed org.

B5. Retrieval-Augmented Generation (RAG), piece by piece

RAG = fetch relevant text first, then give it to the model so it answers from facts rather than memory. Break it into its parts:

1. **Chunking.** Split documents into passages (a few hundred tokens). Too big: noisy and costly; too small: loses context. (Production setting here: **[N]**; the harness default is 512-token passages [O].)
2. **Embedding.** Convert each chunk to a list of numbers (a vector) so that *similar meaning ≈ nearby vectors*. A different embedding model = a different "map of meaning."
3. **Vector search.** Embed the question, find the nearest chunks. Good at *meaning*, weak at exact terms (IDs, rare names).
4. **Keyword search.** Classic term matching. Good at *exact terms*, weak at paraphrase.
5. **Hybrid search.** Run both and merge. Here the NLD index is **hybrid** (vector + keyword) [O].
6. **Pre-filter vs post-filter.** *Pre-filter* narrows the candidate set *inside* the search engine (cheap, safe). *Post-filter* removes results *after* retrieval (can leave you with too few). Here both exist [O/C].
7. **Reranker.** A second, more accurate model re-orders the top candidates. Here: up to **320 chunks**, **12 s** budget [O]. If it fails, original order is kept [O].
8. **top-k and threshold.** How many results to return, and a minimum similarity. Default topK **10** (max 2048) [C]. For tool selection: **top 5 at 0.7 similarity** [C].
9. **Recall vs precision.** *Recall*: did the right item make it into the results at all? *Precision*: how much of what came back is relevant? With a top-5 funnel, **recall@5 is the critical number**: if the right tool isn't in the five, the model can't pick it and there is no fallback I found.

Two separate retrievals in this system: - **Document retrieval**: find the NLD documents about the task. - **Tool retrieval**: find which actions are relevant (embeddings bundled in the code jar; top 5; 0.7) [C].

A subtle and important link: each NLD carries a `tools:` list. When a document is selected, its tools become the agent's *enabled tools* [C]. **Retrieval decides which actions the agent may use.** So retrieval quality is partly a *permissions* concern, not just a quality one.

B6. Identity, tenancy and "runs as the user"

Multi-tenancy: one platform serves many customers (tenants/orgs) and must never mix their data or permissions.

JWT: a signed token carrying *claims* (who you are, which org, what you may do, when it expires). Anyone can read it; only the issuer can sign it, so the receiver can trust the claims.

OrgJWT: a short-lived token scoped to an org, issued by Core at the bootstrap step [C/S].

Token exchange: trade one token for another with a different audience or scope: "I'm org X, give me a token that the knowledge tenant will accept." This is how a customer org's Core asks the shared knowledge tenant for results [C].

"Runs as the user": when the action finally executes in Core, it executes with *the requesting user's own permissions*, not a super-powered service identity [C/S]. So even if everything upstream were fooled, the user cannot do more than they could do by hand.

What this means for the knowledge index: it is **one shared tenant** that holds **no customer data** [C/S]. So "tenant isolation" for *knowledge* is not about separating data; it is about *authenticating who is asking* via token claims. Isolation matters for *customer-data* retrievers (other agents), which are separated per org and data space [C].

What breaks: if a token is stolen it works until it expires; a revoked user keeps working until the token lapses, unless re-checked at action time. The exact lifetime and re-check behaviour: **[N]** here.

B7. Service-to-service plumbing

mTLS (mutual TLS): both sides prove their identity with certificates; traffic is encrypted. Used between internal services so that "who is calling" is cryptographically known [O].

Service mesh (Envoy sidecars): a proxy next to each service handles mTLS, retries on some errors, and timeouts, so the application code doesn't. Inside one domain traffic is plain HTTP that the mesh upgrades to mTLS [O].

SSE (server-sent events): one HTTP connection kept open; the server pushes events as they happen. Used to stream the model's output to the browser. The chain is SSE all the way: browser → Agent API → session engine → planner → reasoner [O].

Why streaming matters: LLM answers take seconds to minutes. Streaming shows progress, but it introduces *long-lived connections* with their own timeouts (here a **601 s** cap) [O].

What breaks: a long-lived connection crosses many hops, each with its own timeout. If those don't line up, the user sees an error even though the backend finished. This is a real defect in this system (Part F).

B8. Reliability: timeouts, retries, idempotency, circuit breakers

- **Timeout:** give up waiting after N seconds. Needed so one slow dependency doesn't hold resources forever.
- **Retry:** try again after a failure. Safe only if repeating the call has no extra effect.
- **Idempotent:** doing it twice has the same effect as once. *Reads* are naturally idempotent. A *create* is not, unless you give it a unique request key so the server can say "already did that."
- **Circuit breaker:** after repeated failures, stop calling the sick dependency for a while and fail fast or use a fallback.
- **Rate limit:** cap requests per tenant to protect shared capacity (here **200 requests / org / minute**) [O].
- **Partial failure:** in a multi-step plan, step 1 succeeds, step 2 fails. Without a plan for it you get half-done state.

The key insight for this system: the *LLM gateway retries up to 10×* [O] because model calls have no side effects. The *action path has no retries at all* [O], because a blind retry of "create group" could create a duplicate. **The correct order is idempotency first, retries second.** Knowing why, not just that, is the senior answer.

B9. Human-in-the-loop (HITL)

A privileged write should require a person's explicit consent. In this system there are **layers** of it [C/S]:

1. **Plan approval**: the user approves the whole plan (single-task plans auto-approve).

2. **Tool confirmation**: a per-action flag (`requireUserConfirmation`) plus a deterministic confirmation message from the planner.
3. **Draft-then-commit**: for membership, the system first produces a *preview* (`commitChanges=false`), the user confirms, then it *commits*.

Hard vs soft controls: a rule written in the prompt ("NEVER call without confirmation") is a *soft* control (the model might not obey). A flag enforced by code, or a deterministic message emitted by the planner, is a *hard* control. Defence in depth means soft controls are a backup, not the guarantee.

The gap: at least one generic write tool has no confirmation, and single-task plans skip plan approval [C]. Consent is therefore *non-uniform*. That is a design weakness you can name.

B10. Delivery: how code reaches users safely

- **Release train:** changes ship on a fixed schedule with the whole product, not whenever ready. Here, Core ships on a train; the agent graph is promoted **every Tuesday** [O].
- **Feature gate:** the code is deployed to everyone but only *enabled* for some orgs/users. Rollout is by *who*, not by *what percent of traffic*. [C]
- **Canary:** send a small slice of traffic to the new version first and watch. Used for the services (Argo canary) [O].
- **Blue/gold:** two parallel environments; switch traffic between them for instant rollback [O].
- **Rollback scope:** here the `Stable` switch reverts *only the graph*, not actions, templates or NLDs [O]. Knowing what a rollback does *not* undo is as important as knowing the rollback.

B11. Observability: seeing inside a running system

- **Logs:** records of events. **Structured logs** carry named fields (traceId, tenant, user).
- **Trace / correlation id:** one id carried across every hop so you can reconstruct a request's journey.
- **Metrics / SLIs / SLOs:** an SLI is a measurement ("percent of turns that succeed"); an SLO is a target for it ("99.5%"). Alerts fire when you are burning through the allowed error budget.
- **Here:** logs per layer; planner trace steps including **grounded/ungrounded** checks; session tracing records [C/O]. **No SLOs found** [N], and the **approval pause is not measured** [N].

B12. Evaluating a non-deterministic system

You can't unit-test "the model will behave." You test at the **seams** and measure rates:

- Retrieval: *recall@k*.
- Tool selection accuracy.
- Plan structure validity.
- Executor behaviour (deterministic, so ordinary tests).
- End-to-end golden tasks, scored by rules or by an LLM **judge**.
- **Judge agreement (kappa):** how consistently the judge agrees with itself/humans. Here **κ = 0.649**, moderate; so the pass-rate number is itself noisy [O].
- **Replay of production failures** as regression cases.
- **Release gate:** a rule that blocks shipping if scores drop. Here the suite scores **81.7%** and there is **no gate** [O/N].

Part C. Component by component

Each card: **Job · Why it exists · Holds · Talks to · Fails by · Question to ask yourself.**

C1. Setup chat (the surface)

- **Job:** the chat window inside Setup. Collects the message, shows streamed text, plans, confirmation cards, result cards.
- **Why:** the admin needs a place to talk and to *approve*. The approval UI is a safety control, not decoration.
- **Holds:** nothing durable; it is a client.
- **Talks to:** Core bootstrap (to get a token), then the Agent API (to stream).
- **Fails by:** no token → can't open a session; stream timeout → false error banner even though the backend succeeded.
- **Ask yourself:** which hop first sees the user's identity, and what proves it?

C2. Bootstrap servlet (the token mint)

- **Job:** in Core, turns "this browser has a logged-in Setup session" into a short-lived **OrgJWT**.
- **Why:** the services outside Core need a token they can verify without trusting the browser.
- **Fails by:** it is the **root of trust** for the whole flow, so how it validates the session matters (exact validation: **[N]**). Internal callers that used an old path hit 404s after a path change [S].
- **Ask:** what does everything downstream assume about this token?

C3. Agent API (`bot-svc-api`)

- **Job:** the public front door for agent conversations. Accepts the message, checks the OrgJWT, rate-limits, opens the SSE stream.
- **Why:** one controlled entry point for all agent traffic.
- **Fails by:** 401 bad token, 429 rate limit; the 601 s stream cap [O].
- **Ask:** why does port 443 need a JWT while the internal port uses mTLS?

C4. Session engine (`bot-svc-worker`)

- **Job:** owns the conversation session; keeps session state in Redis; hands turns to the planner.
- **Why:** someone has to remember the conversation across turns. **This is the real home of the "conversation manager" idea**; no service by that exact name exists [N], so the role is split between this and the graph state.
- **Fails by:** session creation returns a misleading 500 when the planner's provisioning is stuck in `ENQUEUED` [S].
- **Ask:** what state would be lost if this restarted mid-plan?

C5. Planner (`bot-svc-llmagent`)

- **Job:** the *runner* of the agent. It picks the planner type (here `Atlas__ConcurrentMultiAgentOrchestration`, set in the agent YAML) and relays between the reasoner and Core: the reasoner says "execute action X" and the planner performs the call into Core.
- **Why:** it is the **seam between thinking and doing**. It emits the *deterministic* confirmation message for tools flagged `requireUserConfirmation` [S], and carries the identity headers into Core (`x-org-id`, `x-sfdc-user-id`, tenant id, app context) [S].
- **Fails by:** action timeouts (60 s, 90 s in one region), no retries, failures published as events [O].
- **Ask:** which component is actually allowed to *touch* Core, and why is it not the reasoner?

C6. Reasoner "Atlas Daisy" (the graph runner)

- **Job:** executes the agent graph and calls the LLM at each step through the LLM gateway.
- **Why:** it is where the *thinking* lives, isolated from where the *doing* lives.
- **Holds:** graph state in Redis [O].
- **Load:** about **5 LLM calls per request at P95, 12 at P99**; rate-limited to **200 requests/org/minute** [O]. LLM calls dominate cost and latency.
- **Ask:** if every request costs ~5 model calls, what happens to cost and latency as users grow?

C7. The four graph nodes

- **Knowledge node:** calls `QueryDataSource` to retrieve documents; the model picks relevant ones; their `tools:` become enabled tools. If a retrieval branch fails it returns nothing and the node *continues with fewer documents or no tools* [C], which is **silent degradation**.
- **Assistant node:** builds the plan (a todo list). The turn **ends** and the plan is shown. Single-task plans auto-approve [C].
- **Executor node:** runs the approved plan's actions, one per step, using the confirmation flags.
- **Discovery node:** finds records/metadata the plan refers to.
- **Ask:** at which node can a mistake still be caught by a human, and at which can it not?

C8. LLM Gateway (`bot-svc-llm`) and the Trust Layer

- **Job:** one controlled path to model providers (OpenAI, Azure, Anthropic, Bedrock).
- **Why:** central place for *provider failover, rate limits, safety checks, region rules, auditing*.
- **Trust Layer checks:** PII, toxicity, prompt-defence (about 2 s each) [O].
- **Resilience:** circuit breaker fails OpenAI over to Azure; a kill switch routes to a fallback region; up to 10 retries [O]. **But** it is a single point of failure (a 40% availability dip was seen [S]) and the Core side has no circuit breaker [C].
- **Ask:** why can this layer retry aggressively when the action path cannot?

C9. Core actions (where the change really happens)

- **Job:** the code that checks and performs the change, inside Core, *as the user*.
- **The chain for CreateGroup:** REST `/support/functions/createGroup` → mapped to an action type → **adapter** (requires Manage Users or delegated admin) → **validator** → **permission-check class** (403) → save the group record [C].
- **Membership:** `ManageUserAssignments` → draft (`commitChanges=false`) → user confirms → commit (max 50, bulk save) [C].
- **Allowlist and blocklist:** the action is on the copilot allowlist with a named owner; `CREATE_GROUP` is *blocked* from public REST/Apex surfaces so only the agent path reaches it [C].
- **Fails by:** validation/permission errors; **partial results** (group created, members failed) because there is no compensation [I].
- **Ask:** which checks here would still protect you if the model were fully compromised?

C10. Retriever and AI Gateway (`C360Retriever`, `einstein-ai-gateway`)

- **Job:** Core's retriever calls the AI Gateway, which queries Data Cloud, then reranks.
- **Auth:** token exchange into the knowledge tenant; one retry after a refresh [C].
- **Budgets:** retriever query **35 s**, reranker **12 s** [O].
- **Ask:** what does "retriever failure = empty branch" do to answer quality, and would a user ever know?

C11. Data Cloud (the knowledge store)

- **Job:** holds the indexed NLD and help documents; serves hybrid search.
- **Objects, in order:** *data stream* (the feed) → **DLO** (raw landed data) → **DMO** (modelled data) → **chunks** → **embeddings** → **hybrid index** → **retriever** (a named, configured query) [O].
- **Tenant: one shared tenant** (`agentforcenld`), no customer data; every org reads it [C/S]. Backup tenant exists.
- **Ingestion: manual**, no schedule; a corpus change also needs a code-jar version bump, about **12–24 h** to reach Core [O].
- **Risk:** one shared tenant in one region; repeated search-deadline timeouts seen [S].
- **Ask:** if this tenant is slow, whose requests suffer?

C12. The NLD corpus (the knowledge itself)

- **What an NLD is:** a Markdown document describing how to do one task via the APIs, plus metadata (entities, permissions, `tools:`). Types: Higher-Order (which direction), Workflow (steps), Best-Practices.
- **How it's made:** LLM-assisted drafting from help docs, API specs and action descriptions, then **human review**, then Markdown files in Core, collected from about 15 team branches into CSV in S3 [HLD/O].
- **Why it's a risk surface:** documents steer production behaviour (including which tools are enabled), yet the only stated quality control is human review; evals are advisory.
- **Ask:** if one NLD has a wrong step, how many orgs does it affect, and how fast can you roll it back? (Rollback: re-upload and rebuild [I].)

Part D. One request in slow motion

Request: "Create a public group *Sales Ops* and add Ravi and Meera." Follow *control* (who is acting) and *state* (what is stored where).

#	What happens	Who has control	State / what's stored	What could go wrong
1	Admin types the message	Browser	Nothing durable	—
2	Chat asks Core for a token	Core bootstrap	OrgJWT issued, short-lived	No/expired token: chat can't open
3	Chat opens the stream with `Bearer OrgJWT`	Agent API	Rate-limit counters	401, 429
4	Session created, message handed on	Session engine	**Session state in Redis**	Provisioning stuck → misleading 500
5	Planner selected, request relayed	Planner	Request context headers	Planner down alert; 4xx/429
6	Graph starts; model classifies and prepares retrieval	Reasoner + LLM gateway	**Graph state in Redis**	Model/provider failure; rate limit
7	Knowledge node asks `QueryDataSource`	Reasoner → planner → Core	—	Action timeout, no retry
8	Core runs doc/NLD/nav branches in parallel; retriever → AI gateway → Data Cloud hybrid search → rerank	Core + AI Gateway + Data Cloud	Index (shared tenant)	Search deadline hit → empty branch → fewer docs, silently
9	Core filters by permission; model picks docs; chosen docs' `tools:` enable actions	Core + model	Enabled-tools list in graph state	Wrong doc → wrong tools
10	Assistant node builds a 2-task plan; **turn ends**; plan is shown	Reasoner → user	`current_plan_shown` in graph state	Plan wrong/hallucinated; user abandons (no expiry found)
11	Admin approves; arrives as a **new message**	User → graph	`todo_approved` set	Org changed since the plan was made (stale plan)
12	Executor calls `createGroup`	Reasoner → planner	—	—
13	Planner emits the **deterministic confirmation message**; admin confirms	Planner ↔ user	—	Double prompts / loops reported
14	Planner calls Core `/support/functions/createGroup`	Planner → Core	—	60 s timeout, **no retry**; 403 from permission/JWT
15	Core: adapter checks Manage Users → validator → permission check → **save `Group`**, *as the user*	Core	**Group row created in the org DB**	Validation error; duplicate name
16	Group summary returns to the graph	Core → reasoner	graph state updated	—
17	`ManageUserAssignments` runs as a **draft**	Core	nothing committed yet	Wrong users resolved
18	Admin sees the preview, confirms	User	—	—
19	Commit: bulk save of `GroupMember` rows (max 50)	Core	**Members rows created**	**Failure here leaves group without members (partial)**

#	What happens	Who has control	State / what's stored	What could go wrong
20	Result card streams back over SSE	Core → … → browser	—	**Slow turns (≈4.5–5 min) can show a false error even though it worked**

Three things to notice 1. **The write only happens at steps 15 and 19, inside Core, as the user.** Everything before is advice and consent. 2. **Consent appears three times** (plan, tool confirmation, draft-commit). That is defence in depth, but not uniform across tools. 3. **Control never goes from the model straight to the database.** It always passes through the planner and Core's checks.

Part E. Why it is built this way, and where it trades off

For each choice: **the reason**, **what you gain**, **what you give up**. Items marked **[I]** are my reading, not stated in the sources.

Choice	Reason	Gain	Cost
Graph instead of a ReAct loop	Enforce *retrieve → plan → approve → execute* by structure	Predictable, pausable, testable control flow	10–20% slower (Jan 2026 doc)
Plan shown before execution	A human consents to a *specific* plan	Accountability; fewer surprises	One extra turn; single-task plans skip it
Separate planner from reasoner	Keep the component that thinks away from the one that touches Core	Narrow blast radius	More hops, more timeouts, no retries
Knowledge as Markdown documents	Add capability without code	Scales to the long tail; non-engineers can contribute	Documents steer production; quality control is human review; manual ingestion
Retrieval chooses tools (`tools:`)	Keep the model's menu small	Accuracy and cost	Retrieval becomes a permissions concern; recall@5 risk
Actions run in Core as the user	Reuse the platform's permission model	Cannot exceed the user's own rights	Core is a bottleneck/latency path
Agent definition ships in the Core jar **[I]**	Gating per org/user, access to Core data, release train	Consistency with Core, easy gating	Coupled to Core's weekly cadence; rollback covers graph only
One shared knowledge tenant	Knowledge is global, holds no customer data	Simple, one index to maintain	Single point of failure, single region
LLM gateway with retries, actions without	Model calls are side-effect-free; writes are not	Resilience where safe	Action failures surface to the user
Generic CRUD actions	Cover the long tail	Fewer tools to build	Bigger capability surface; needs uniform gates

Design document vs the code (important for honesty)

- The doc describes a **ConversationManager supervisor**; the code splits that role (session engine + graph state).
- The doc describes **periodic** metadata extraction with a status machine; the code shows **manual** ingestion and no such MDLO in this path.
- The doc describes a **confidence score**; the code has none that gates execution (the constants are dead code).
- The doc's **100% fidelity** executor is the goal; in practice approval is enforced in several places and not uniformly.
- Parallel execution, streaming, generative UI and Prism discovery are **proposals**.

Part F. How to diagnose a failure (symptom → layer)

Work from the user's symptom to the layer, then ask the layer's question. This is how to *tell an incident story* in the interview.

Symptom	Most likely layer	First question	Typical evidence
Chat won't open / "can't start session"	Token or session creation	Did bootstrap issue an OrgJWT? Is planner provisioning stuck?	Bootstrap logs; 401/500 at the Agent API
"Something went wrong" but the change happened	**Timeout chain**	Did the turn run ≈4.5–5 min?	Backend success logs vs stream cap and mesh timeouts
Agent answers vaguely / ignores the right workflow	**Retrieval**	Did the document branch return anything? Is the right doc in the top results?	Retrieval logs; empty branch; `DEADLINE_EXCEEDED`
Agent picks a wrong action	Tool selection	Was the right action in the top 5 above 0.7?	Tool-retrieval scores
Plan looks wrong	Reasoning (LLM)	Which doc steered it? Was a clarification skipped?	Planner trace steps; grounded/ungrounded flag
Action fails with 403	**Permission/identity**	Does the user hold Manage Users? Was the JWT valid?	Core action log; permission-check class
Group exists but no members	**Partial failure**	Did the commit step fail or time out?	Action log for `ManageUserAssignments`
Everything slow or failing at once	Gateway/provider	Is the LLM gateway or a provider degraded? Is the org rate-limited (429)?	Gateway metrics; circuit-breaker state
Stale or wrong instructions	**Knowledge freshness**	When was the corpus last uploaded and the index rebuilt?	Ingestion records; jar version
Works in some orgs only	Gating/provisioning	Is the org/user gate on? Is provisioning complete?	Access-check config

The habit to build: name the *layer* (token / session / retrieval / planning / tool-API / auth / cache / infra), then the *mechanism*, then the *evidence that confirmed it*, then the *fix and the regression test*. That is the template for your challenging-problem story.

Part G. Self-test: explain it back (coaching drills)

Cover the pages. Answer aloud in your own words, then check. If you can't, re-read the named part. **Tell me which you struggle with and I will quiz you on those.**

Foundations (Parts A–B) 1. State the v2 idea in five sentences. (A3) 2. Why can't a model be trusted to act directly, and what sits in between? (B1) 3. Bespoke vs generic action: what do you gain and lose? (B2) 4. ReAct loop vs graph: what is enforced by structure? Relate it to LangGraph. (B3) 5. Walk the RAG parts and say which single number matters most for a top-5 funnel. (B5) 6. Why does knowledge retrieval double as a permissions concern? (B5) 7. What does "runs as the user" guarantee, and what does it not? (B6) 8. Why does the LLM gateway retry 10× but the action path not at all, and what must exist before action retries are safe? (B8) 9. Hard vs soft controls: give an example of each in this system. (B9) 10. What does the `Stable` switch undo, and what does it not? (B10)

System (Parts C–D) 11. Which component first sees the user's identity, and how is it proven downstream? (C1–C3) 12. Why is the planner the "seam between thinking and doing"? (C5) 13. What happens, step by step, if one retrieval branch times out? (C7, C10) 14. Name the three places consent appears in create-group. (D) 15. Where exactly does the data change happen, and under whose rights? (C9, D) 16. What is shared across every org in the knowledge path, and what is not? (C11)

Judgement (Parts E–F) 17. Pick two places the design doc and the code differ. How would you present them? (E) 18. A user says "it errored but the group exists." Diagnose it. (F) 19. A user says "the agent used outdated instructions." Diagnose it. (F) 20. If the agent were compromised, what bounds the damage, and where is the bound weaker than intended? (B9, C9) 21. You inherit this system: what do you fix first and why? (E, plus the gaps list in the architecture PDF)

Out loud, timed: the 60-second system overview, the 2-minute request walk-through (Part D), and one failure story (Part F) using the template.

Part H. Glossary

Term	Plain meaning
Agent	A system that uses a model to decide steps and call tools toward a goal
Agent graph	The control flow drawn in advance as nodes and edges with shared state
Atlas Daisy	The reasoning service that runs the graph
Planner	The runner that relays between the reasoner and Core and emits deterministic confirmations
Session engine	Owns the conversation session and its state
Action / tool	A named, schema-typed function the agent may call
Bespoke / generic action	One-task tool / broad parameterised tool
NLD	Natural Language Document: a Markdown description of how to do a task via APIs, with metadata
RAG	Retrieve relevant text first, then answer from it
Embedding / vector	Numbers capturing meaning so similar text sits close together
Hybrid search	Vector search plus keyword search, merged
Reranker	A more accurate model that re-orders top candidates
topK / threshold	How many results / minimum similarity
recall@k	Whether the right item appears in the top k
Chunk	A passage-sized piece of a document
DLO / DMO	Raw landed data object / modelled data object in Data Cloud
Data stream	The feed that brings source data into Data Cloud
Retriever	A named, configured search query over an index
Tenant / org	One customer's isolated slice of the platform
JWT / OrgJWT	Signed token with claims / a short-lived org-scoped one
Token exchange	Swapping one token for another with different audience/scope
mTLS / mesh	Mutual certificate auth between services / proxies that handle it
SSE	One long-lived HTTP connection the server streams events over
Redis	In-memory store used here for session and graph state
HITL	Human-in-the-loop: a person approves before a privileged action
Draft-then-commit	Preview a change, confirm, then apply
Idempotent	Doing it twice equals doing it once
Circuit breaker	Stop calling a failing dependency; fail fast or fall back
Release train	Fixed-schedule release of everything together
Feature gate	Deployed everywhere, enabled for selected orgs/users
Canary / blue-gold	Small-slice rollout / two parallel environments to swap
SLI / SLO	A measurement / a target for it
Trust Layer	Checks around model calls: PII, toxicity, prompt defence
Judge / kappa	An LLM scoring outputs / how consistently it agrees

Term	Plain meaning
Long tail	The many rare tasks too numerous to hand-build

Agentforce for Setup v2: Architecture, Gaps, and Interview Q&A

Prepared for: GitLab, Senior SWE Agentic SDLC, round with Karan Bhatia, 2026-10-09 **Built from:** (1) the v2 High-Level Design (Jan 2026), (2) the code-and-Slack trace of the live system (branch `p4/264-patch`, 2026-10-04) **Example request used throughout:** "Create a public group *Sales Ops* and add users X and Y."

> **CONFIDENTIAL — study material only.** Contains internal component names. In the interview, speak at the **pattern level**, round numbers, no repo paths, no ticket IDs, no Slack quotes. Three open security findings from the trace are **deliberately omitted** from this document and should not be raised in the interview.

How to use this document

1. **Sections 1–7:** learn the system. Be able to draw Section 1 on a whiteboard from memory.
2. **Section 8:** the gaps. These are your most senior-sounding material: *you found where the design and the code differ.*
3. **Section 9:** about 30 likely questions with answer spines. Answers describe **the system**. Wherever it matters, there is a `YOUR ROLE:` slot. Fill it only with what is verified (OWN / REVIEWED / CONSUMED / NONE).
4. **Section 10–12:** your role sentence, the challenging-problem template, and questions to ask Karan.

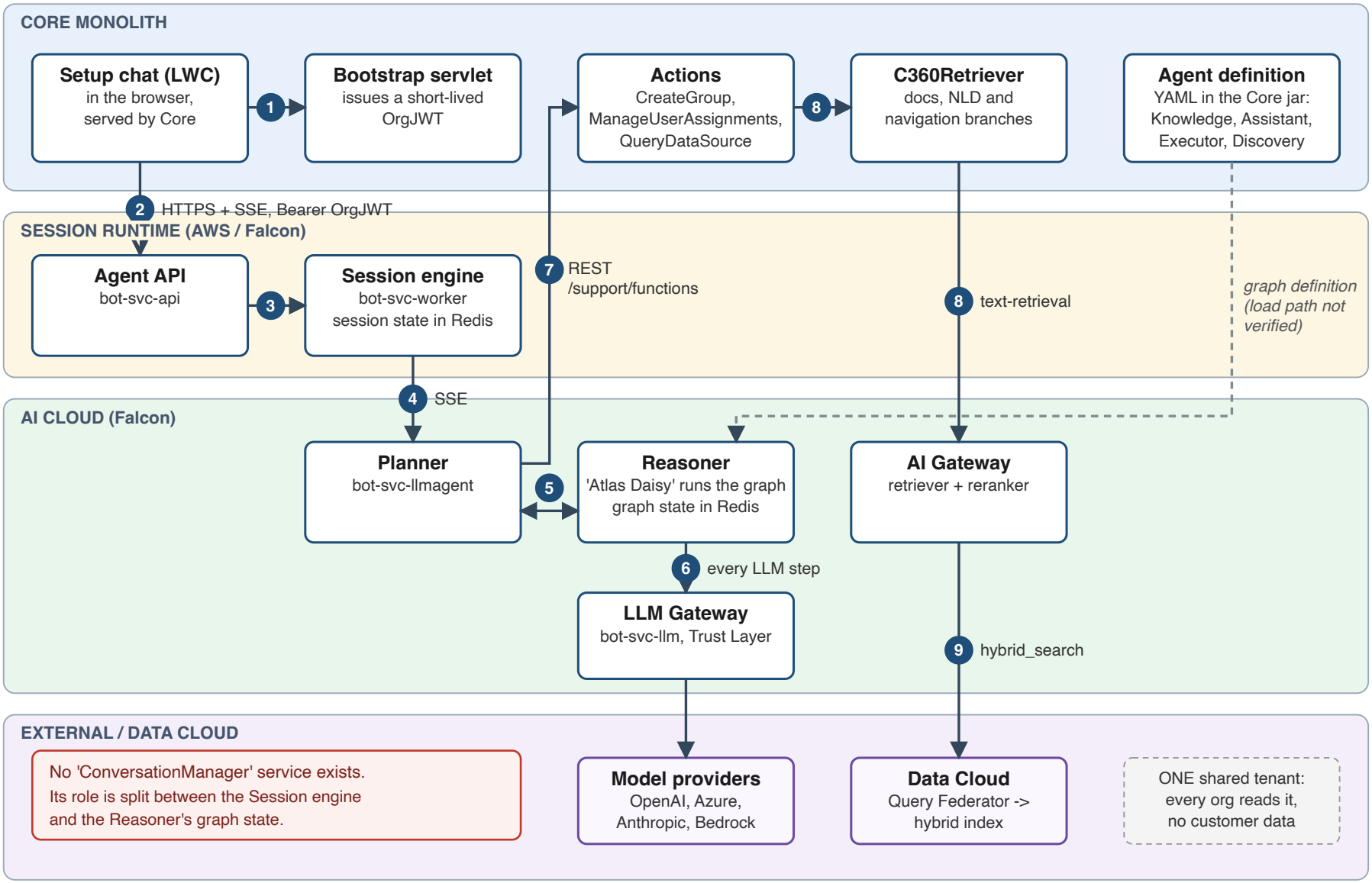
Evidence tags: **[C]** verified in the Core repo · **[O]** verified in an off-core service repo · **[S]** stated in Slack, not verified in code · **[I]** inference · **[N]** searched for, not found.

Ground rules for every answer - Never claim authorship of anything not verified in your commits/PRs. "I traced it and here is how it works" is true and strong; "I built it" is only for things you built. - When a fact is **[N]** or **[I]**, say so: "I didn't find that in the code" / "I'm inferring." - Proposals in the design doc (parallel execution, streaming, generative UI, Prism discovery) are **not** shipped features.

---

1. Architecture on one page

Figure 1. Architecture by platform. Numbers match the hop table in Section 2



Component map

Role (design-doc term)	What it is in the code	Runs on
Surface	Setup chat LWC + streaming client	Core monolith
Agent runtime	Agent API (`bot-svc-api`) + session engine (`bot-svc-worker`)	Internal runtime on AWS (Falcon)
Conversation manager	**No service by this name [N].** Role is split: session state (worker, Redis) + graph state (reasoner, Redis)	as above
Planner	`bot-svc-llmagent`	AI Cloud (Falcon)
Reasoning	`agent-svc-reasoner` ("Atlas Daisy")	AI Cloud (Falcon)
Knowledge / Assistant / Executor / Discovery	Nodes in the agent graph, defined in a YAML file shipped **inside the Core jar**	executed by the reasoner
Knowledge content	NLD Markdown files in Core + a knowledge repo	Core + S3
Retriever	Core `C360Retriever` -> AI Gateway	Core + AI Cloud
LLM Gateway	`bot-svc-llm` (model calls, Trust Layer)	AI Cloud
AI Gateway	`einstein-ai-gateway` (retrievers, reranker)	AI Cloud
Data Cloud	One shared tenant (`agentforcenld`): data stream, DLO, DMO, chunk, hybrid index, Query Federator, search engine	Data Cloud
Actions	`CreateGroup`, `ManageUserAssignments`, `QueryDataSource`, `FilterKnowledgeDocuments`	Core monolith

One-line mental model: the *agent definition and the actions* live in **Core**; the *reasoning loop and LLM access* live in **AI Cloud**; the *knowledge* lives in **Data Cloud**; the user session sits on an **AWS-hosted runtime**.

---

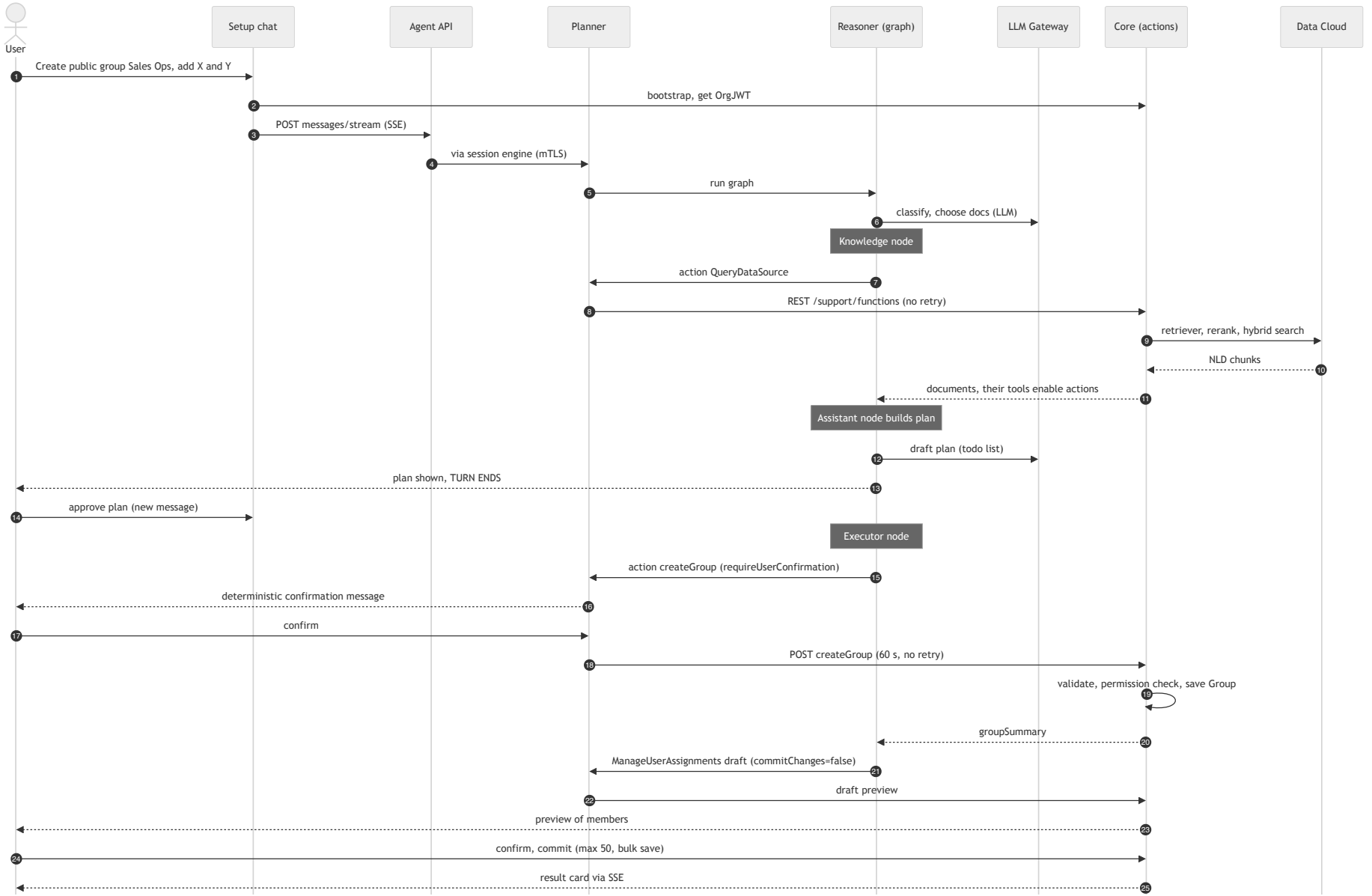
2. Request flow: hop by hop

Same five questions per hop: protocol, auth, timeout, retry, failure.

Hop	What happens	Auth	Timeout / retry	On failure
1 Bootstrap	Chat asks Core for a token	User's Core session -> **OrgJWT**	none set / none	No token, chat can't open a session
2 Surface -> Agent API	HTTPS POST, SSE stream	`Bearer OrgJWT` (port 443); internal 9443 uses mTLS	**stream cap 601 s**; no client timeout / no retry	401 bad JWT, 429 rate limit
3 Agent API -> worker	HTTP upgraded to mTLS by the mesh	mesh mTLS	not found [N]	Session create returns 500 (common cause: planner provisioning stuck)
4 Worker -> planner	SSE	mTLS	not found [N]	"Initializing session failed", 429
5 Planner -> reasoner	HTTP over mesh	mesh mTLS	planner->reasoner timeout [N]; reasoner retries mesh 503s during rolling deploys	**GRLS rate limit 200 req/org/min**; about 5 LLM calls per request at P95, 12 at P99
6 Reasoner -> LLM Gateway	HTTPS	mesh mTLS + tenant id header	reasoner connect 5 s / read 90 s; gateway per-provider 25 s (Bedrock 90 s); Trust Layer checks 2 s each; mesh 300 s. **Gateway retries up to 10x**; Core side 2 tries in prod, no circuit breaker	OpenAI circuit breaker fails over to Azure; kill switch to us-west-2; a 40% availability dip has been seen
7 Knowledge node -> QueryDataSource	action via planner into Core	Bearer from token exchange	60 s (90 s in one region); **no retry**	Failed branch returns nothing; agent continues with fewer docs or no tools (silent)
8 Core -> AI Gateway	`POST .../retrievers/{name}/text-retrieval`	**Token exchange** into the knowledge tenant	retriever 35 s, reranker 12 s; 1 retry after token refresh	Reranker failure keeps original order; retriever failure = empty branch
9 AI Gateway -> Data Cloud	SQL `hybrid_search` / `vector_search`	tenant-scoped JWT	search ~11 s deadline, query service 60 s	`DEADLINE_EXCEEDED` -> empty retrieval (seen repeatedly on the NLD index)
10 Plan approval pause	Plan shown, **turn ends**; approval arrives as a *new message*	same as 2–5	no expiry found in v2 [N]; state in Redis, TTL [N]	No approval = no side effect

Hop	What happens	Auth	Timeout / retry	On failure
11 Executor -> action -> Core	reasoner -> planner -> Core REST	reasoner->planner: no auth header (mesh); planner->Core: Bearer from token exchange, exact grant [N]	reasoner->planner 60–90 s; planner->Core 60 s (90 s one region); **no retry on either leg**	`ProxyResponseError`; 403 surfaces as "Invalid response from Core Action Execution"
12 Core action	in-process Java; runs as the represented user	user context from the JWT	Core request limits [I]	validator/permission errors, 401/403; **partial result possible** (group exists, members missing) [I]
13 Return path	SSE in reverse	same connections	mesh 300 s, stream 601 s	**False "Something went wrong" on 4.5–5 min turns even though the change succeeded**

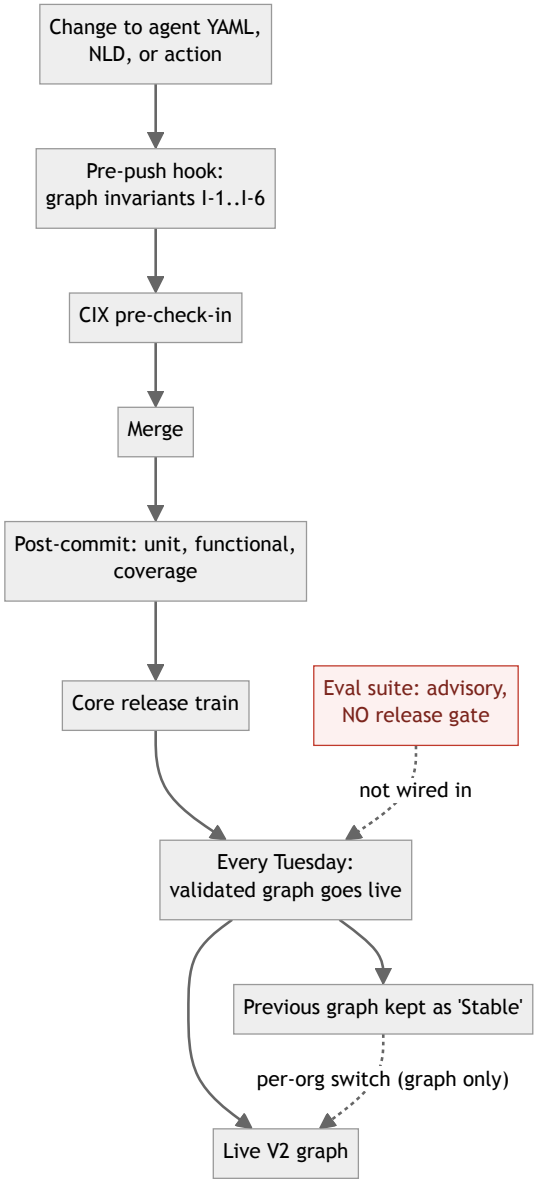
Figure 2. Sequence: create group Sales Ops and add X and Y



3. Where it runs, and how it ships

Deployable	Build	Gates	Release	Rollback
Core modules (agent YAML, NLDs, actions, validators)	Bazel modules aggregated into the Core app	CIX pre-check-in pipeline, then post-commit unit tests / functional tests / coverage. Pre-push hook blocks edits to the agent YAML until graph invariants I-1..I-6 pass	Core release train. **Agent graph promoted every Tuesday** by a rotating owner; previous graph becomes `Stable`	Per-org switch to the `Stable` graph. **Reverts the graph only, not actions, templates, or NLDs**
NLD corpus (knowledge)	CI build to artifact repo	evals are advisory, not a gate	bump jar version in Core, then **manual** S3 upload and **manual** index rebuild on 4 orgs	re-upload previous corpus [I]
Agent API / session engine	Strata CI -> Docker image	service CI	Helm + Terraform add-ons, FD-staged promotion with **Argo canary**	Argo / Helm rollback
Planner	Strata CI	service CI	soak instance first, then broader; **blue/gold releases**; Argo canary; freeze around Dreamforce	switch blue/gold, Argo rollback
Reasoner	Strata CI	service CI	mesh-staggered rollout (~21 instances)	Argo rollback
LLM Gateway	Strata CI	health-check analysis during rollout	FedX deploy with a 1-minute pre-rollout pause	automated on failed analysis
AI Gateway	Strata CI	fit tests	Falcon Helm	Helm rollback

Figure 3. Release train for Core changes and the agent graph



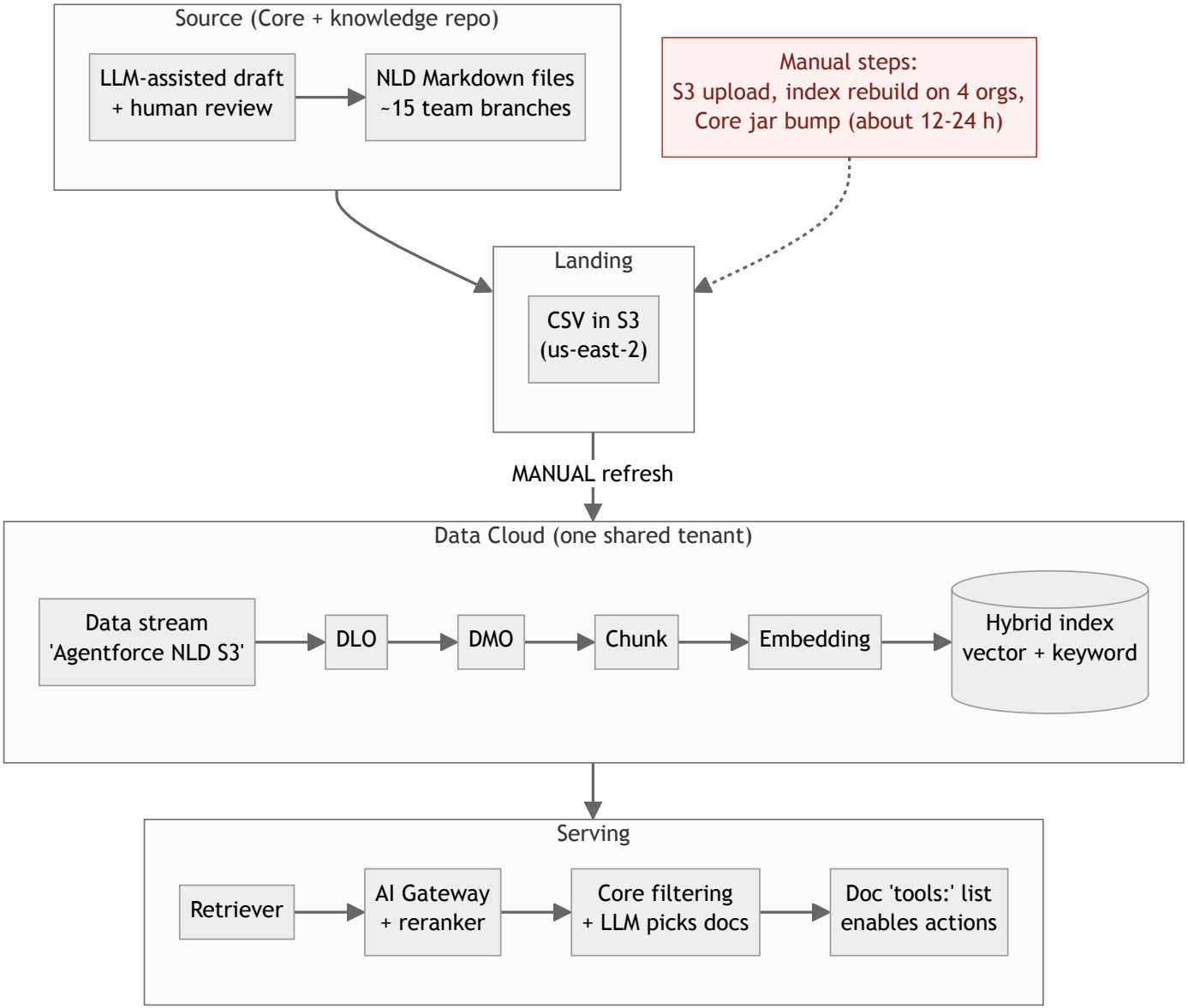
Autoscaling data points: LLM Gateway HPA 3–9 (4 CPU / 12 Gi); AI Gateway HPA 3–12. Redis for session/graph state is a managed AWS service; its definition file was not located [N].

Release-train mechanics worth knowing: feature gated per org and per user (org permission + gate, user permission), shipped to every instance with the Core release, enabled progressively. That is progressive delivery **by feature gate**, not by traffic percentage.

---

4. Data Cloud path (knowledge)

Figure 4. Knowledge ingestion and retrieval pipeline



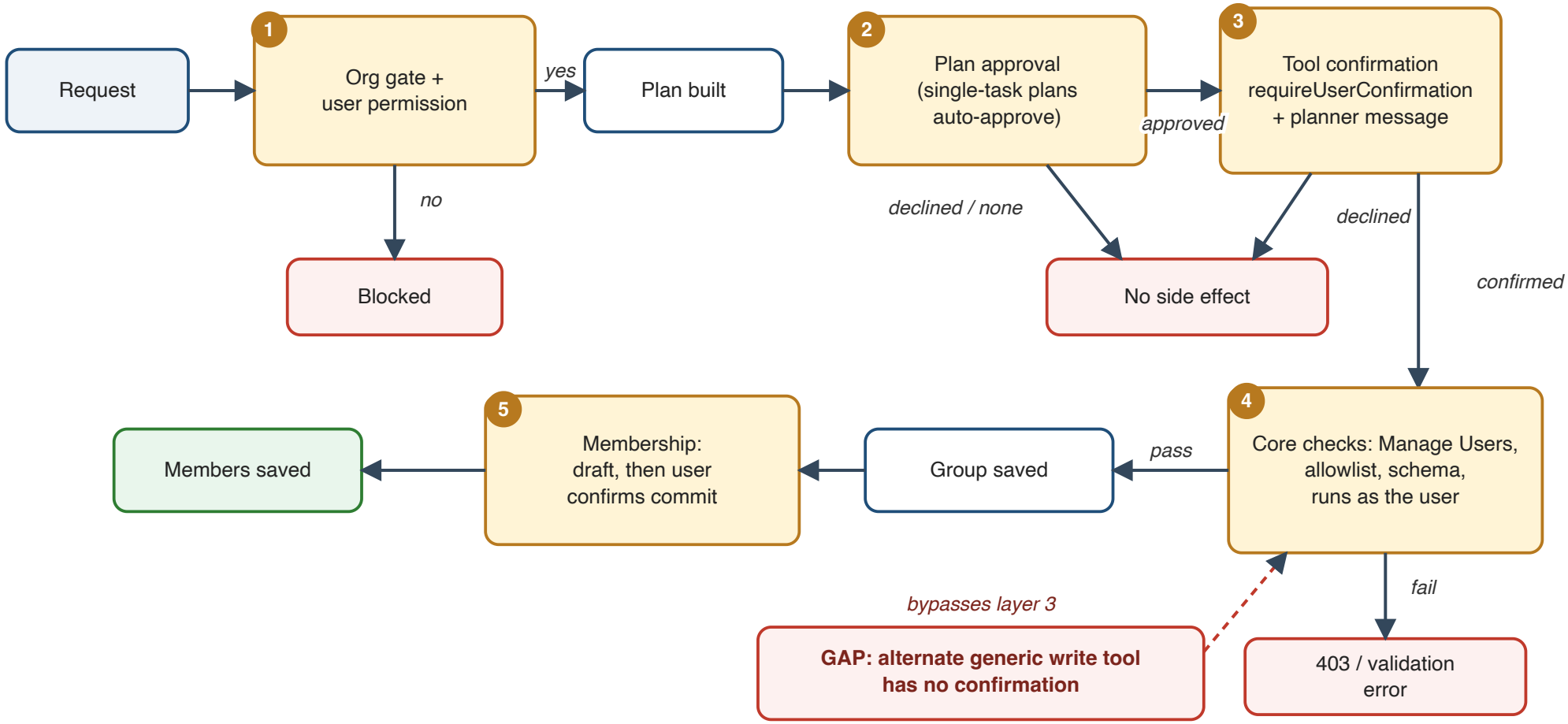
Item	Finding
MDLO / status machine from the design doc	**Not present in this path [N].** Corpus lands directly as a DLO from an S3 data stream
Ingestion cadence	**Manual / skill-driven, no schedule.** Refresh the data stream, rebuild the index on 4 orgs; a corpus change also needs a Core jar bump (~12–24 h)
Tenant model	**One shared Salesforce-owned tenant**, holds no customer data; every org reads the same index; caller identified by token claims. Customer-data retrievers (other agents) are isolated per org and data space via signed claims (`NO_DATASPACE_ACCESS`)
Pre-filter	Filter expression passed into `hybrid_search` (runs inside the search engine)
Post-filter chain	reranker (up to 320 chunks, 12 s) -> optional post-processor (off) -> Core filtering (permissions, client, de-dup) -> LLM selects docs -> `FilterKnowledgeDocuments` keeps only those
topK	default 10, max 2048
Tool retrieval (separate path)	**top 5**, similarity threshold **0.7**, embeddings bundled in the jar
Chunking / embedding in production	**Not found [N].** (Harness defaults: 512-token passages, a 1024-dim embedding, cosine, HNSW)
Fusion weights / score threshold	Not found [N]

Interesting design detail: each knowledge document carries a `tools:` list. When the LLM selects a document, its tools populate `enabled_tools`. **Knowledge retrieval therefore controls which actions the agent can use** — retrieval quality is also a *permissions-surface* question.

---

5. Safety and approval: the layers

Figure 5. Approval and permission layers on the write path



Layer	Mechanism
Org / user gating	Org permission + gate, user permission `UseSetupWithAgentforce`, plus capability checks
Action permission	`CreateGroup` requires Manage Users or delegated admin, plus a permission-check class returning 403
Runs as the user	Core executes under the represented user's normal permissions
Allowlist	Action is on the copilot action allowlist with a named owner
Surface blocklist	`CREATE_GROUP` is **blocked** from the public REST and Apex invocable surfaces; reachable only via the agent path
Metadata write map	`Group` : CREATE / READ / UPDATE only
Schema validation	Input schema with `unevaluatedProperties: false` + server-side validator
Human approval (4 layers)	(1) plan approval, single-task plans **auto-approve**; (2) per-tool `requireUserConfirmation` + deterministic planner confirmation message; (3) **draft-then-commit** for membership; (4) future runtime with pre-tool hook (auto-declines after 5 min, lost on restart)
Prompt-level	Rule "NEVER call without confirmation" (a *soft* control)
Trust Layer	PII, toxicity, prompt-defense checks (2 s each), guardrails timeout 10 s

6. Observability and evaluation

Logs and traces: LLM timing log (traceId, template, generation id, durations, token counts, errors); action log (action, status, input, output, time, target id); planner trace steps (UserInput, LLM, Function, Transition, PlannerResponse, **OutputEvaluation GROUNDED / UNGROUNDED**); session tracing DMOs; Zipkin-style traceId in Core; OpenTelemetry sampled at 1% in the LLM Gateway.

Splunk: separate indexes per layer (Core, runtime, planner, retriever). Harness lookup covers **preprod only**.

Alerts found: retriever availability (Sev2 < 99.7%, Sev1 < 99.3%, **Slack only, PagerDuty off**); planner down alert (thresholds unknown); Data Cloud Query Federator (Sev2 < 96%, PagerDuty). **None found** for the LLM Gateway, reasoner, worker, or the Setup agent itself. **No SLOs found [N].**

Approval pause: no dedicated trace event or metric; pause length and abandonment are **not measured [N].**

Evals: 524-utterance suite, latest 5-run pass rate **81.7%** (88.3% stable); daily backend regression, 499 rows across 53 actions, partly LLM-judged; UI runs vision-graded; judge agreement **κ = 0.649**; create-group scenarios exist in the suite. **No CI gate on pass rate [N]**; the larger gold suite is recommended, not required.

7. Design document vs the code

Design doc says	Code shows	Say this
A persistent ConversationManager supervises specialists	No such service; role split across session engine and graph state	"The design describes a supervisor role; in the implementation it's split."
Ingestion runs periodically via a metadata-extraction framework with a status machine	Manual, no schedule, no MDLO in this path	"The doc describes periodic extraction; what I found is manual ingestion."
Per-tenant orgJWT, "centralized C360"	One shared global tenant with no customer data	"Knowledge is global, so tenancy is about *calling org identity*, not data separation."
Plan gets a confidence score	**No calibrated gate;** threshold constants are dead code	"The doc mentions it; I verified there's no confidence gate."
Executor runs plan with "100% fidelity", no LLM	Confirmation spread across YAML, prompt rule, and planner message; at least one alternate write tool has no confirmation	"Fidelity is the goal; in practice approval is enforced in several places and not uniformly."
RAG funnel narrows actions to 3–5	**Confirmed:** top 5 at 0.7 similarity	State it confidently.
Parallel execution, streaming, generative UI, Prism discovery	Proposals	"Proposals, not shipped."

8. Gaps and risks (with how you would fix them)

The security-sensitive findings from the trace are excluded. The remaining list is ordered by what you would raise first. **The "how I'd fix" column is your own engineering opinion** (fine to present as such).

#	Gap	Evidence	Why it matters	How I'd fix it
1	**Partial side effects, no compensation.** Group created, member step fails or times out	Core action path [C/I]	Orphan group; retry may hit a duplicate-name error	Idempotent create-or-get keyed by a request id; record each step's outcome; report exactly what was done; offer resume or undo
2	**No retries on action execution**	reasoner->planner and planner->Core [O]	A latency blip becomes a failed turn	Add retries **only after** idempotency keys exist; retrying non-idempotent writes is worse than failing
3	**Timeout budgets don't line up** (60 vs 90 s, mesh 300, stream 601)	[C/O]	False "Something went wrong" on 4.5–5 min turns although the change succeeded	One end-to-end deadline propagated downward, each hop gets less than its parent; heartbeat events on the stream
4	**Approval is inconsistent.** Single-task plans auto-approve; an alternate metadata-write tool can create a Group without confirmation; double prompts and loops reported	[C/S]	Side effect without explicit consent; user fatigue	One **deterministic** confirmation gate in the executor/planner for *every* write tool, not prompt text
5	**No eval release gate** (81.7% pass, κ 0.649, weekly promotion)	[O]	Regressions can ship on Tuesday	Block promotion on a per-category pass-rate floor; replay production failures; calibrate the judge to higher agreement
6	**Manual knowledge ingestion** (S3 upload + 4-org rebuild + jar bump, 12–24 h)	[O]	Stale or wrong instructions; tool drift	Scheduled/triggered ingestion with versioned corpus, eval gate before index swap, canary index, fast rollback
7	**Rollback covers only the graph**	[O]	A bad action, template, or NLD can't be reverted by the Stable switch	Version actions and NLDs with the graph as one release unit
8	**Shared single-region knowledge index** with repeated search timeouts	[S]	One Data Cloud incident degrades Setup knowledge for every org; residency question	Multi-region read replicas / failover; local cache of hot documents; explicit degraded-mode behavior
9	**Silent degradation:** a failed retrieval branch returns empty and the agent keeps going	[C]	Quality drops with no signal	Emit a degraded flag in the trace and tell the user when knowledge was unavailable
10	**Monitoring gaps:** no SLOs; retriever alerts Slack-only; no alerts on the LLM Gateway, reasoner, or the Setup agent; approval pause unmeasured	[O/S/N]	Outages found by customers first	SLIs: turn success rate, plan-approval rate, time-to-approve, p95 latency per hop; page on burn rate
11	**LLM Gateway is a single point of failure;** no Core-side circuit breaker	[S/C]	Every turn fails during a gateway incident	Circuit breaker + fast-fail + user-facing degraded message; multi-provider failover

#	Gap	Evidence	Why it matters	How I'd fix it
12	**Provisioning fragility:** planner provisioning stuck in `ENQUEUED` produces misleading 500s	[S]	New orgs/sandboxes can't start sessions	Surface the real provisioning state; add a health check
13	**Dead confidence constants**	[C]	False sense of a confidence gate	Delete them, or replace with verifiable signals (entities resolved, inputs bound, schema-valid, doc matched)
14	**Model/region constraints:** a model move may block a small share of orgs; in-region enforcement vs the us-west-2 kill switch conflict	[O]	Outage or residency conflict for regulated orgs	Per-region model capability matrix and a policy that decides which wins
15	**Zero-data-retention not confirmed for Setup**	[S]	Blocks security reviews	Document and verify retention for RAG context, plans, and history

Prioritisation if asked "what do you fix first?" (1) approval uniformity + idempotency (correctness and safety of writes), (2) eval release gate (stop regressions shipping), (3) observability/SLOs (so you can see the rest). Everything else follows from being able to measure.

---

9. Likely questions and answer spines

Format: **Q** -> **spine** (the structure of a good answer) -> **watch-out**. Answers describe the system. Add `YOUR ROLE:` only where verified.

A. Architecture and design

Q1. Walk me through the architecture end to end. Spine: 30-second version, then the diagram. "The user's request goes through the Setup chat to an Agent API and session engine, to a planner, to a reasoning graph with four nodes: Knowledge retrieves, Assistant builds a plan, the user approves, Executor runs the actions. Actions execute inside the Core monolith as the user. Reasoning and LLM access live in AI Cloud; knowledge lives in Data Cloud." Then name the 13 hops only if asked. Watch-out: don't name a "ConversationManager" service; say "the supervisor role."

Q2. Where is each component deployed, and why there? Spine: three tiers. Core monolith (UI, agent definition, actions, because they need Core data and permission model and ship with the release train). AI Cloud on Falcon (planner, reasoner, gateways — GPU/LLM-adjacent, independently scalable, staged Helm releases). Data Cloud (shared knowledge index). Session runtime on AWS-hosted Falcon. Watch-out: the "why" for putting the agent YAML in the Core jar is **[I]** (per-org gating, coupling to Core data/permissions, release train). Say "my read is..."

Q3. Why separate thinking from doing? Spine: the human approves a specific plan; if the executor re-reasons, what ran may not be what was approved. Deterministic execution makes it testable, auditable, and reproducible. Watch-out: in the real system the guarantee is not absolute (see Q8, Q21). Say so.

Q4. Walk through one concrete request: "create a public group and add two users." Spine: classification -> knowledge retrieval (two docs: CreatePublicGroup, ManageUserAssignments, whose `tools:` enable the actions) -> plan with two tasks -> user approves -> `createGroup` with a confirmation -> Core validator and permission check (Manage Users) -> `Group` saved -> membership is **draft then commit**: preview, user confirms, bulk save (max 50) -> result card. Watch-out: partial failure between the two steps (Q9).

Q5. How does the agent choose among hundreds of actions? Spine: two stages. Vector search over the action catalog returns the **top 5** above a **0.7** similarity threshold, then the LLM picks among five. Knowledge docs also carry a `tools:` list, so retrieval narrows the allowed tool set. Watch-out: top-5 recall is the real risk. If the right action isn't in the five, there's no fallback I found.

Q6. Information-seeking vs action-seeking: why classify first? Spine: different blast radius (read vs write), cost, and UX. The read path can't write, so misclassifying toward "information" is safe; the dangerous direction merely costs latency.

Q7. Generic tools vs bespoke tools: trade-off? Spine: bespoke = narrow, testable, bounded; generic = coverage and low maintenance but the capability surface becomes the whole API. Mitigations: runs as user, entity allowlist, schema validation, uniform confirmation. Watch-out: the real system still has a generic write path (`ExecuteMetadataCreate`) with weaker confirmation than the bespoke path — a live example of the risk.

Q8. If this agent were compromised right now, what's the worst it can do? Spine: bounded by (1) the invoking user's permissions, (2) the action allowlist, (3) the surface blocklist, (4) approval layers. **Be honest:** the bound is weaker than intended because one generic write tool lacks confirmation and single-task plans auto-approve. Fix: one deterministic gate for all writes.

B. Failure handling and reliability

Q9. What happens if the member step fails after the group is created? Spine: result is partial (group exists, members missing); no compensation. A user retry could hit a duplicate name. Fix: idempotent create-or-get, per-

step recording, honest report, resume/undo.

Q10. Why are there no retries on action execution? Spine: writes aren't idempotent, so blind retries risk duplicates. The right order is **idempotency keys first, then retries.** Reads can be retried safely. Watch-out: contrast with the LLM Gateway, which retries up to 10x, because model calls are side-effect-free.

Q11. Your timeouts don't line up. What's the consequence? Spine: action timeout 60 or 90 s, mesh 300 s, stream cap 601 s, no Core default. Result: turns around 4.5–5 minutes show an error although the backend succeeded. Fix: propagate one deadline downward; heartbeats on the stream; make the UI reflect actual state.

Q12. How do you handle LLM provider failure? Spine: circuit breaker fails OpenAI over to Azure, a kill switch routes to a fallback region, the gateway retries, the rate limiter returns 429 above the per-tenant limit. Gap: no Core-side circuit breaker, and the gateway is a single point of failure.

Q13. What happens if retrieval fails? Spine: the branch returns empty and the agent continues with fewer documents or no tools. That's silent degradation; I'd emit a degraded flag and tell the user.

Q14. Where does the human-approval pause live, and what if the user never approves? Spine: the turn **ends**; approval returns as a new user message; graph state sits in Redis. No approval = no side effect. No expiry or TTL found, and pause length is unmeasured. Compare the future runtime: auto-declines after 5 minutes, lost on restart. Watch-out: this is exactly the "durable pause/resume" topic; relate to your own design **only if** you built one.

C. Data Cloud / knowledge

Q15. How does retrieval work end to end? Spine: Knowledge node calls `QueryDataSource` -> Core runs product docs, NLD, and navigation branches in parallel -> `C360Retriever` -> AI Gateway (35 s) -> Data Cloud hybrid search -> reranker (12 s, up to 320 chunks) -> Core filtering -> LLM selects documents -> only those are kept and their tools enabled.

Q16. Walk me through ingestion. Spine: NLD Markdown in Core (drafted with LLM help, human-reviewed) -> collected into CSV in S3 -> data stream -> DLO -> DMO -> chunk -> embedding -> hybrid index. **Manual,** no schedule. Watch-out: the design doc describes a periodic extraction framework; the code shows manual. Say which is which.

Q17. How fresh is the knowledge? Spine: bounded by a manual process: data stream refresh + index rebuild, and a Core jar version bump of roughly 12–24 hours. Near-real-time indexing only once the data has landed. That is the weakest freshness link.

Q18. How is tenant isolation enforced? *(Karan's specialty — prepare this one best)* Spine: two cases. (a) The Setup knowledge corpus is one shared Salesforce-owned tenant containing **no customer data**; isolation isn't about data separation, it's about authenticated caller identity via exchanged token claims. (b) Customer-data retrievers are isolated per org and data space via signed claims, with `NO_DATASPACE_ACCESS` for unauthorized users. Pre-filter runs inside the search engine; post-filters in Core. Bridge: "I'd propagate traceId, tenantId, and user id through every node and across the approval pause."

Q19. Why Data Cloud rather than your own vector store? Spine: reuse of existing search infra, ingestion, and governance. Counter-risk: a single shared region and repeated search deadline timeouts. **Watch-out:** the "why" is **[I]**.

Q20. What retrieval parameters are used, and were they tuned? Spine: topK default 10 (max 2048), reranker up to 320 chunks, tool retrieval top-5 at 0.7. Production chunking, embedding model, and fusion weights were **not found [N]**; don't state them. Whether they were tuned with eval data is unknown.

D. Deployment and operations

Q21. How does a change reach production? Spine: by deployable. Core modules: Bazel build, pre-check-in pipeline, post-commit tests, release train; the agent graph is promoted **every Tuesday** and the prior graph

becomes `Stable`. Services: CI -> Docker -> Helm/Terraform -> staged promotion with Argo canary or blue/gold. Knowledge: manual. `YOUR ROLE:` which of these pipelines you personally touched. Only that.

Q22. How do you roll back? Spine: per-org switch to the `Stable` graph (fast, safe). Limit: **graph only**; actions, templates, and NLDs aren't reverted. Services: Argo/Helm rollback or blue/gold switch.

Q23. What does your release process miss? Spine: no eval gate on promotion; manual knowledge ingestion; rollback scope; freeze periods. Fix: gate promotion on category pass-rates and replay of production failures.

Q24. How do you scale it? Spine: stateless services behind HPA (gateway 3–9, AI Gateway 3–12), state in Redis, rate limit per org (200 req/min), roughly 5 LLM calls per request at P95 and 12 at P99 — LLM calls dominate cost and latency, so token and call budgets matter more than CPU.

Q25. What do you monitor? How do you know it's healthy? Spine: logs per layer, planner trace steps, grounding outcome, session tracing. Alerts exist for the retriever, planner, and Data Cloud federator. Gaps: no SLOs, no alerts on gateway or reasoner, approval pause unmeasured. I'd define SLIs: turn success, approval rate, time-to-approve, per-hop p95.

E. Quality and evaluation

Q26. How do you test a non-deterministic planner? Spine: test at seams: retrieval recall@k, tool-selection accuracy, plan structure, executor determinism (pure code), end-to-end golden tasks, and replay of production failures. Real numbers: 524-utterance suite at 81.7% pass, judge agreement κ 0.649, no CI gate. `YOUR ROLE:` the eval framework on your resume — verify what part of *this* system it covers before linking them.

Q27. How do you know the knowledge documents are correct? Spine: LLM-drafted, human-reviewed; that's the only stated control. Add: evals as a release gate, versioning, canary index.

F. Judgement

Q28. What's the single biggest risk? Pick one and defend it: *non-uniform approval over a partially non-idempotent write path.* One tool without a confirmation, auto-approved single plans, and no compensation on partial failure means a side effect can happen with weaker consent than users assume.

Q29. You inherit this system. What do you fix first and why? Spine: Section 8 prioritisation: write safety (uniform approval + idempotency), then an eval release gate, then observability/SLOs. Rationale: you can't safely improve what you can't measure, and you can't measure safely while writes are inconsistent. Note: this is close to Karan's own situation, new to a platform with documented risks.

Q30. Where did the design and the implementation diverge? Spine: Section 7. Pick two: ingestion (periodic in doc, manual in code) and confidence score (in doc, absent in code). Say how you found out: you traced the design against the code.

Q31. Two admins create the same group name at once. Spine: duplicate-name behavior in the validator wasn't confirmed **[N]**. Reason: don't rely on the model to coordinate; enforce uniqueness and conflicts in the data layer, return a clear error, and make the agent's retry idempotent.

Q32. Is retrieved content or org metadata a prompt-injection path? Spine: yes in principle; any text the LLM reads can carry instructions. The Trust Layer includes prompt-defense checks, but I didn't verify injection testing specific to this agent **[N]**. The strongest mitigation is structural: writes require approval and run as the user, so injected text can't act without a gate (which is why gate uniformity matters).

---

10. Your role sentence (fill in once verified)

Do not use any of these until the office-Claude ledger confirms it.

If the ledger shows	Say
You authored actions/NLDs/eval cases in the access-management area	"I owned [specific actions/NLDs/eval cases]; the platform team owned the runtime and planner."
You only consumed/read it	"I wasn't on the planner. I traced this system closely because my own access-management agents use the same pattern, and here's how I'd improve it."
Mixed	State each piece separately with a verb that matches the evidence (authored / reviewed / consumed).

Never say: "I worked on the planner" or "I wrote entities for the generic actions" unless a commit or PR proves it.

11. "Tell me about a challenging problem": template

Pick **two or three** from work you can prove. Fill every blank with a fact; if you can't, the story isn't ready.

```
1. Context and stakes (one sentence, one number)
2. Symptom: how it showed up (alert / report / metric, timestamp)
3. First hypothesis, and why it was right or wrong
4. How you narrowed it (the exact query / log / trace)
5. Root-cause LAYER: retrieval / planning / tool-API / auth-token / cache / infra
6. Options considered, what you rejected, why
7. Fix, and how you verified it (metric before/after, with method)
8. What you'd do differently; the regression or eval case you added
9. What YOU did vs what teammates / the platform team did
```

Probes to rehearse on each story: "What was the alternative?" "How do you know it worked?" "Blast radius if you were wrong?" "What did you personally do?" "How would you have caught it earlier?"

---

12. Questions to ask Karan

- "Where's the biggest gap between the public Duo Agent Platform vision and what's shipped today?"
- "When an agent opens MRs at machine rate, does governance live in the platform or per agent?"
- "How do you think about per-tenant isolation for agent context and memory on a shared platform?"
- "Can you trace one MR from the triggering signal through every tool call and the human-approval pause?"

---

13. Still unverified: say "I didn't find it," never guess

How the bootstrap servlet validates the session · the exact planner->Core token-exchange grant · timeouts on the agent-API->worker, worker->planner, and planner->reasoner hops · Redis TTL for graph and approval state · production chunking, embedding model, fusion weights, and thresholds · SLOs and several alert thresholds · Setup-specific zero-data-retention confirmation · whether a recent planner confirmation change applies to this agent · duplicate-name handling in the validator · prompt-injection testing for this agent.

14. Numbers cheat sheet

Number	Meaning
45	bespoke actions in the v1 pilot (design doc)
Top 5 / 0.7	tool retrieval candidates / similarity threshold
10 (max 2048)	retriever default topK
320 / 12 s	reranker chunks / timeout
35 s	gateway retriever query timeout
~11 s / 60 s	search deadline / query service deadline
60 s (90 s one region)	action timeout reasoner->planner->Core
300 s / 601 s	mesh timeout / stream cap
25 s (Bedrock 90 s)	LLM Gateway per-provider timeout
10 retries	LLM Gateway retry count (no retry on actions)
200 / org / min	rate limit
~5 and 12	LLM calls per request at P95 and P99
81.7% / 88.3% / κ 0.649	eval pass / stable / judge agreement
Tuesday	weekly graph promotion
12–24 h	knowledge corpus change reaching Core
50	max membership assignments per call
10–20%	v2 slower than v1 planner (design doc, Jan 2026; not re-verified)